

WEEK5_SUMMARY

Week 5 Summary - Cardiac Patient Monitoring API

Chosen Project and Scope

The Phase 3 capstone is a Healthcare Management API focused on cardiac patient monitoring, including patient profiles, vital signs, medications, appointments, and secure access for administrators, doctors, and patients. By Week 9, the project will provide a documented REST API and Postman collection, a normalized SQL Server database with EF Core migrations and an ERD, JWT authentication with role-based access control, critical unit and integration tests, CI/CD, deployment, and a complete README. The scope is realistic because the core domain and security baseline already exist, allowing Sprints 1-4 to focus on consolidating core routes, strengthening authentication and performance, and completing testing, documentation, and deployment.

Day 5 Risk Priorities

Test coverage	Why it is high risk	High-risk logic	Priority
Unknown-user rejection, valid JWT/refresh-token issuance and persistence, and rejection of refresh-token reuse	A failure could allow unauthorized access .or token replay	Authentication and refresh-token rotation	1
An owning patient receives 200 OK ; a different patient receives 403 Forbidden ; service interaction is verified with .Moq	A failure could expose one patient's medical record to .another patient	Patient ownership authorization	2
Valid boundary values pass; out-of-range heart rate, oxygen saturation, systolic BP, and diastolic .BP all fail	Invalid clinical values could be stored and used in medical .decisions	Clinical vital-sign validation	3

Test Suite

The suite uses xUnit, Moq, EF Core InMemory, and `WebApplicationFactory<Program>`.

	Scenarios	Type	Test group
	Heart-rate calculation, invalid age, and blood-pressure classification	Unit	<code>CardiacCalculatorTests</code>
	Login failure, token issuance/persistence, and refresh-token replay prevention	Unit	<code>AuthServiceTests</code>
	Repository return value, exception path, mapping, and interaction verification	Unit with Moq	<code>PatientServiceTests</code>
	Mock return/exception paths plus patient ownership allow/deny paths	Unit with Moq	<code>PatientsControllerTests</code>
	Valid boundaries and all invalid clinical ranges	Unit	<code>CreateVitalSignRequestValidatorTests</code>
	Authenticated happy path, not found, and unauthenticated access	Integration	<code>PatientsApiTests</code>
	Safe standardized <code>ProblemDetails</code> response	Integration	<code>GlobalExceptionMiddlewareTests</code>
			Total

Full-suite command

Run from the `CardiacMonitor` directory:

```
dotnet test .\CardiacMonitor.slnx --configuration Release
```

Latest Day 5 result: **21 passed, 0 failed, 0 skipped.**

Integration-Test Setup

- The application is hosted in memory with `WebApplicationFactory<Program>`.
- SQL Server registration is replaced with an isolated EF Core InMemory database.
- The database is reset before each patient API test.
- The protected patient endpoint is exercised with a JWT obtained from the real `/api/auth/login` route.
- Happy, not-found, and unauthenticated paths are covered.

Error-Handling Setup

- `GlobalExceptionHandler` is registered before the rest of the request pipeline.
- Unhandled exceptions are logged with the HTTP method, request path, and trace ID.
- Clients receive an `application/problem+json` response with a generic title and trace ID.
- The response does not expose the exception message, exception type, or stack trace.
- A non-production diagnostic endpoint verifies the behavior through the full HTTP pipeline.

Sprint 1 Handoff

The project is ready for Sprint 1 planning with a working API skeleton, database model and migrations, core healthcare routes, authentication and RBAC, centralized error handling, and a passing automated test suite. Sprint 1 should confirm the ERD and backlog, close any missing core-route behavior, and treat the current high-risk tests as the minimum regression baseline for every subsequent sprint.